PREPARATION SHEET LAB 04

IIR FILTER IMPLEMENTATION IN MATLAB AND IN C

Team (Group number + letter as chosen in Moodle)	**Team members**	
	1	Emir Avni Sahin
1-1	**2**	Efe Kardes
	3	

Prep task 1: lowpass/highpass filter design

1. Write a MATLAB script *iir_lab.m* that designs an elliptic low-pass filter, a Chebyshev low-pass filter, and an elliptic high-pass filter with the above specifications. Use the MATLAB functions for IIR filter designs *ellipord, ellip, cheb1ord* and *cheby1*. The next steps are to be performed for all three filters respectively.
2. Convert the MATLAB numerator and denominator filter coefficients into second-order sections using the MATLAB function *tf2sos* in this way $[sos, g] = tf2sos(b, a)$
3. For this lab the scaling factor 'g' may be **evenly distributed** over the sections. Integrate the factor into the coefficients.
4. Note that coefficients greater than 1 after accounting for the factor 'g' require appropriate scaling. Since such scaling must be repaired in the implementation in C, it should be divided by 2, for example, but in no case scaled to the maximum.
5. Convert the MATLAB filter coefficients of the second-order sections to *int16_t* values for the DSP implementation using a suitable scaling factor and write them to header files. For this purpose, use a function as specified in Listing 4.1.
6. Plot the amplitude response of the filters on an absolute scale and in dB.
7. Check in particular that the minimum stopband attenuation is maintained and the behaviour in the passband is correct.

For the forthcoming simulations and DSP implementations, use the *int16_t* coefficients of the second-order sections, not the direct form!

Matlab script (part 1-5):

```
clear; clc; close all;

%% 1. LAB SPECIFICATIONS

fs = 50000;       % Sampling Frequency in Hz

fp = 10500;       % Passband edge frequency in Hz
```

```
fs_stop = 16500;   % Stopband edge frequency in Hz

delta_p = 0.01;    % Linear passband ripple

Rs = 40;           % Minimum stopband attenuation in dB

%% 2. CALCULATE DECIBEL RIPPLE & NORMALIZED FREQUENCIES

Rp = -20 * log10(1 - delta_p); % Convert linear ripple to dB

Wp = fp / (fs/2);              % Normalized passband frequency (0 to 1)

Ws = fs_stop / (fs/2);         % Normalized stopband frequency (0 to 1)

% For the High-Pass mirrored at Fs/4 (which is exactly W = 0.5)

% The mirrored frequencies are simply 1 - W_lp

Wp_hp = 1 - Wp;

Ws_hp = 1 - Ws;

fprintf('Calculated Passband Ripple (Rp): %.4f dB\n\n', Rp);

%% 3. FILTER DESIGN ALGORITHMS

% --- Elliptic Low-Pass ---

[n_ellip, Wn_ellip] = ellipord(Wp, Ws, Rp, Rs);

[b_ellip, a_ellip] = ellip(n_ellip, Rp, Rs, Wp);

% --- Chebyshev Type I Low-Pass ---

[n_cheb, Wn_cheb] = cheb1ord(Wp, Ws, Rp, Rs);

[b_cheb, a_cheb] = cheby1(n_cheb, Rp, Wp);

% --- Elliptic High-Pass ---

[n_hp, Wn_hp] = ellipord(Wp_hp, Ws_hp, Rp, Rs);

[b_hp, a_hp] = ellip(n_hp, Rp, Rs, Wp_hp, 'high');

%% 4. SOS SCALING AND QUANTIZATION (Prep Task 1.3 - 1.5)

% We use an anonymous/local helper function (defined at the end of script)
```

```
% to distribute 'g', scale by 0.5 to prevent >1, and quantize to int16.

sos_ellip_sim = process_sos(b_ellip, a_ellip);

sos_cheb_sim  = process_sos(b_cheb, a_cheb);

sos_hp_sim    = process_sos(b_hp, a_hp);

%% 5. GENERATE HEADER FILES

write_coeff_biquad(sos_ellip_sim, 'coeff_ellip_lp.h', 'ellip_lp');

write_coeff_biquad(sos_cheb_sim,  'coeff_cheb_lp.h',  'cheb_lp');

write_coeff_biquad(sos_hp_sim,    'coeff_ellip_hp.h', 'ellip_hp');

fprintf('Header files successfully generated in current directory.\n\n');

%% 6. TIME DOMAIN SIMULATION (Prep Task 2)

n = 0:255;

t = n / fs;

% Input signal: Two cosines at 6 kHz and 17 kHz

xn = 0.5 * cos(2*pi*6000 * t) + 0.5 * cos(2*pi*17000 * t);

% Filter the signal using the INT16 Quantized SOS coefficients

yn_ellip = sosfilt(sos_ellip_sim, xn);

yn_cheb  = sosfilt(sos_cheb_sim,  xn);

yn_hp    = sosfilt(sos_hp_sim,    xn);

%
==========================================================================

% HELPER FUNCTIONS

%
==========================================================================

function sos_sim = process_sos(b, a)

   % 1. Convert to SOS and get gain
```

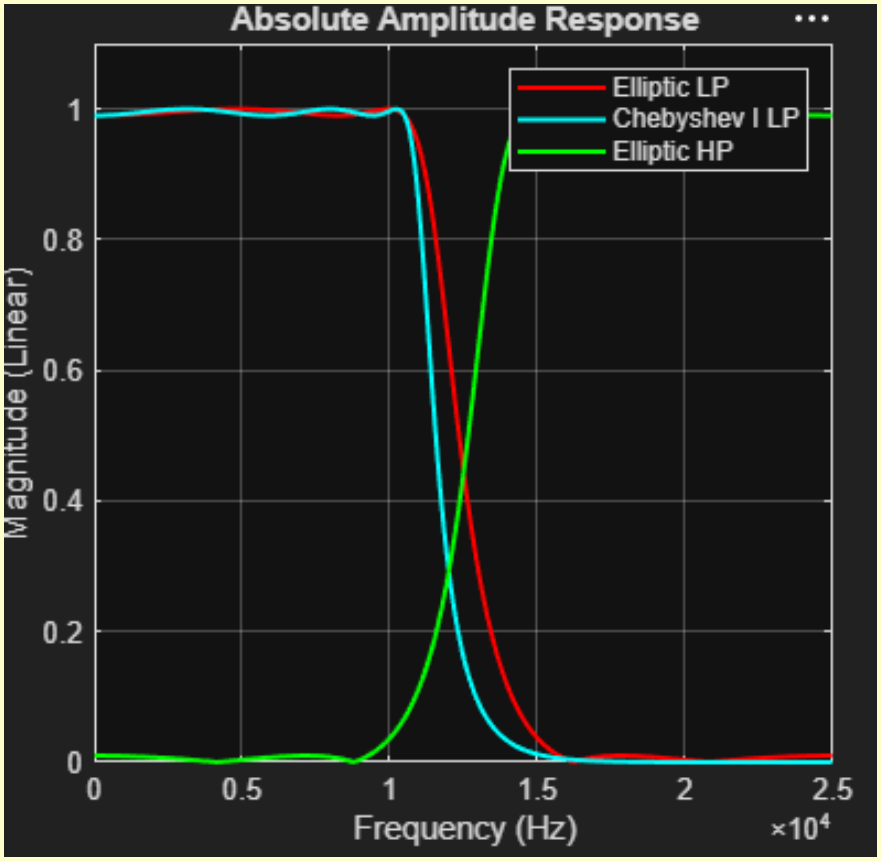
```
    [sos, g] = tf2sos(b, a);
    % 2. Distribute gain evenly
    N_sec = size(sos, 1);
    sos(:, 1:3) = sos(:, 1:3) * (g^(1/N_sec));
    % 3. Check for > 1 and scale by 0.5 if necessary
    if max(abs(sos(:))) >= 1.0
        sos = sos * 0.5;
    end
    % 4. Quantize to 16-bit to simulate hardware exactly
    sos_int16 = round(sos * 32767);
    sos_sim = sos_int16 / 32767;
end
function write_coeff_biquad(sos, filename, array_name)
    % Based on Listing 4.1 from Lab PDF
    filnam = fopen(filename, 'w');
    fprintf(filnam, '#define N_sections_%s %d\n', array_name, size(sos, 1));
    fprintf(filnam, 'int16_t asCoeffIir_%s[%d][%d] = {\n', array_name, size(sos, 1), size(sos, 2));
    for k = 1:size(sos, 1)
        fprintf(filnam, '    {');
        for i = 1:size(sos, 2)
            fprintf(filnam, ' %6d,', round(sos(k, i) * 32767));
        end
        fprintf(filnam, ' },\n');
    end
```

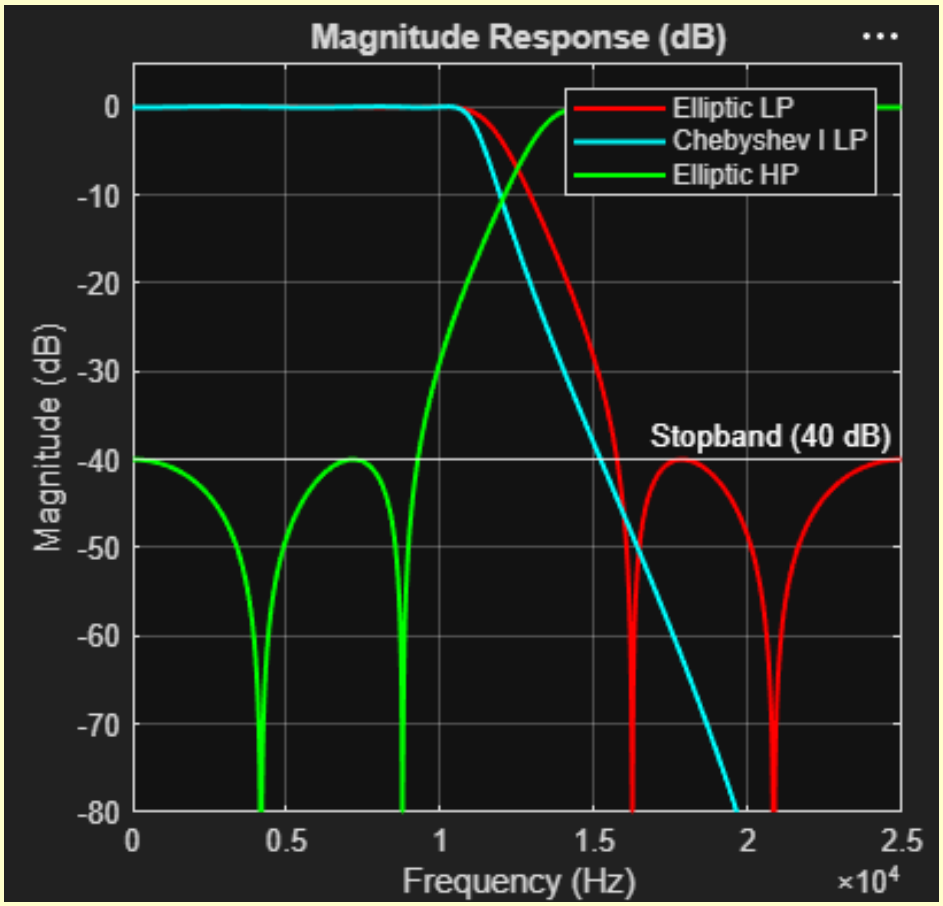
```
    fprintf(filnam, '};\n');

    fclose(filnam);

end
```

Amplitude responses (part 6-7):





Content of header file(s):

```
#define N_sections_ellip_lp 2

int16_t asCoeffIir_ellip_lp[2][6] = {

   {  5185,   8977,   5185,  16384,  -6841,   2541, },

   {  5185,   4683,   5185,  16384,  -3195,  11156, },

};

#define N_sections_ellip_hp 2

int16_t asCoeffIir_ellip_hp[2][6] = {

   {  5185,  -8977,   5185,  16384,   6841,   2541, },

   {  5185,  -4683,   5185,  16384,   3195,  11156, },

};
```

```
#define N_sections_cheb_lp 3

int16_t asCoeffIir_cheb_lp[3][6] = {

  {  3161,   6322,   3161,  16384, -14804,   4278, },

  {  3161,   6337,   3175,  16384,  -9773,   7879, },

  {  3161,   6308,   3147,  16384,  -5500,  13177, },

};
```

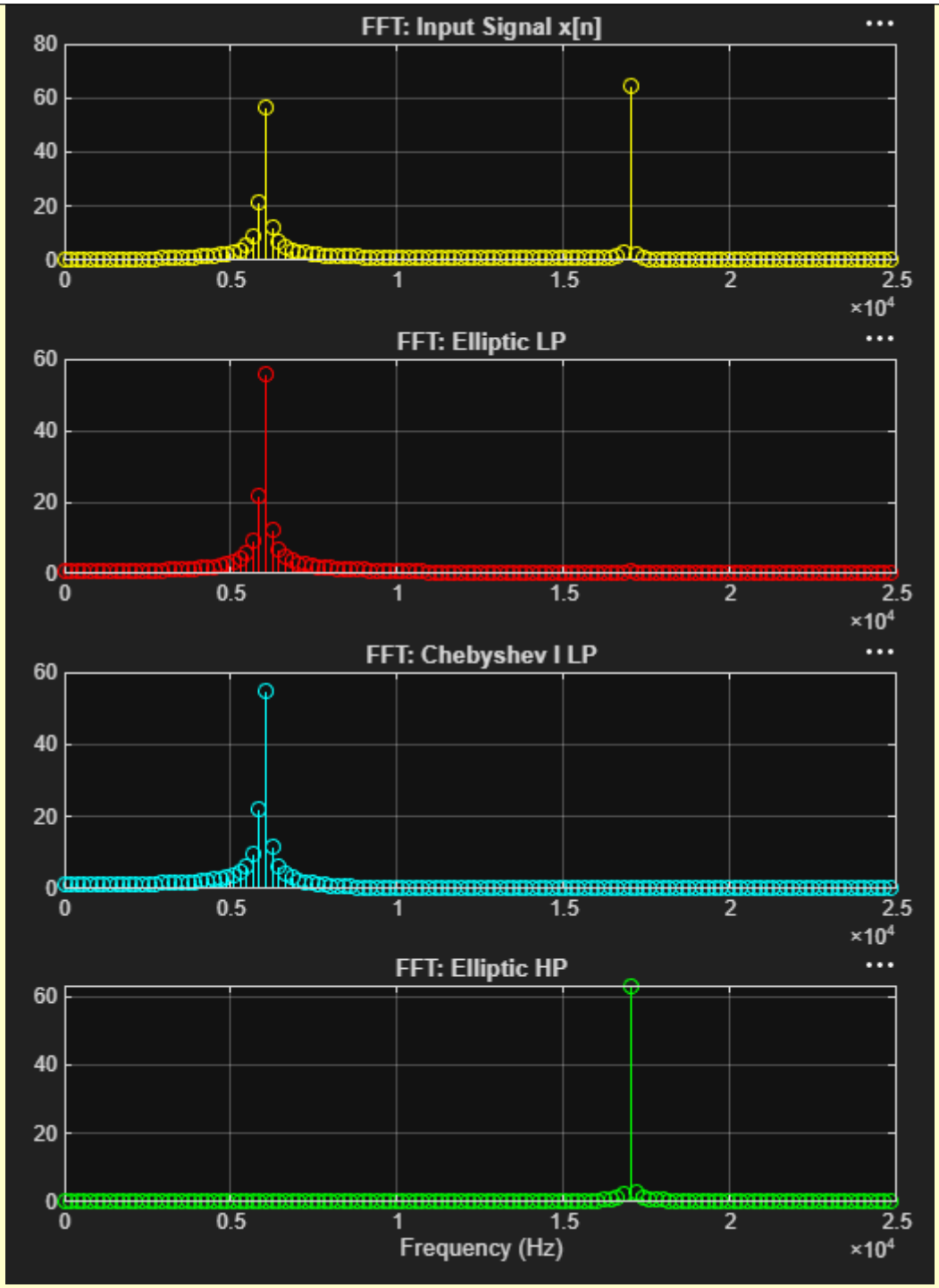
Prep task 2: Time domain simulation

- Extend your MATLAB script: Have the output y_n of all three filters calculated for the input x_n, as described above.
- Plot x_n and y_n (ellip, cheby1, ellip HP).
- Show $abs(fft(x\_n))$ and $abs(fft(y\_n))$.
- Comment your results.

Plots:


Input Signal x[n] (6 kHz + 17 kHz)
Output y[n]: Elliptic LP
Output y[n]: Chebyshev I LP
Output y[n]: Elliptic HP
Samples [n]

FFT Plots:



Comments:

Filter Design Analysis: Both the elliptic and chebyshev type 1 low-pass filters successfully meet the strict specifications: a passband ripple of less than 0.087 dB (linear 0.01) and a stopband attenuation of strictly greater than 40 dB. The Elliptic filter achieves this with a steeper roll-off compared to the chebyshev Type 1 filter. The high-pass filter, created by mirroring the normalized frequencies at Fs/4, successfully flips the passband and stopband characteristics.

Time-Domain Simulation: The input signal x is a superposition of a 6 kHz cosine (in the LP passband) and a 17 kHz cosine (in the LP stopband).

- For both low-pass filters (elliptic and chebyshev), the 17 kHz high-frequency component is heavily attenuated, and the time-domain output y becomes a clean 6 kHz sine wave.
- For the high-pass elliptic filter, the 6 kHz component falls into the stopband and is removed. The output y becomes a clean 17 kHz sine wave.

FFT Analysis: The absolute FFT of the input signal clearly shows two distinct magnitude spikes at 6 kHz and 17 kHz. In the output FFTs of the LP filters, the 17 kHz spike is completely suppressed down to the noise floor, leaving only the 6 kHz peak. The high-pass output FFT shows the exact opposite behavior, suppressing the 6 kHz peak. This validates our Biquad (SOS) hardware simulation coefficients.